



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“FORMULARIO REGISTRO DE CONTACTOS”

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I.	INFORMACIÓN GENERAL	3
-	Objetivos	3
-	Equipos, materiales, programas y recursos utilizados	3
II.	MARCO TEORICO	3
III.	PROCEDIMIENTO	4
	Programa: Catálogo de Productos y Buscador:.....	4
	1. Diseño del Programa.....	4
	2. Tabla de Controles.....	4
	3. Código del Programa.....	5
IV.	ANALISIS E INTERPRETACION DE RESULTADOS	13
	CONCLUSIONES	13
	RECOMENDACIONES	13
	WEBGRAFIA	13



INFORME DE LABORATORIO

TEMA: FORMULARIO REGISTRO DE CONTACTOS

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Desarrollar un sistema de registro y administración de contactos.
 - Implementar operaciones CRUD (Crear, Leer, Actualizar y Eliminar).
 - Aplicar programación orientada a objetos mediante clases.
 - Utilizar listas dinámicas para almacenamiento de información.
 - Implementar validaciones de datos ingresados por el usuario.
 - Aplicar búsquedas dinámicas utilizando LINQ.
 - Utilizar DataGridView para mostrar información estructurada.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

Programación Orientada a Objetos (POO): La programación orientada a objetos permite representar entidades del mundo real mediante clases y objetos. En este sistema, cada contacto se representa mediante la clase Contact.

CRUD: CRUD representa las operaciones fundamentales de un sistema de gestión:

- Create: Agregar registros.
- Read: Consultar registros.
- Update: Modificar registros.
- Delete: Eliminar registros.

LINQ: LINQ permite realizar consultas sobre colecciones de datos de manera sencilla y eficiente.

BindingSource: BindingSource actúa como intermediario entre los datos y la interfaz gráfica, facilitando la actualización dinámica del DataGridView.

Validación de Datos: Las validaciones garantizan que la información ingresada sea correcta antes de almacenarse.

DataGridView: Permite visualizar datos en forma tabular, facilitando la interacción con el usuario.



III. PROCEDIMIENTO

Programa: Catálogo de Productos y Buscador:

1. Diseño del Programa

2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Registro de Contactos
	Font	bold type
Label2	Text	ID Contacto
Label3	Text	Nombre
Label4	Text	Apellido
Label5	Text	Teléfono
Label6	Text	Dirección
Label7	Text	Género
Label8	Text	Buscar
Button1	Name	btnAgregar
	Text	Agregar
Button2	Name	btnModificar
	Text	Modificar
Button3	Name	btnEliminar
	Text	Eliminar
Button4	Name	btnLimpiar
	Text	Limpiar



Textbox1	Name	txtId
Textbox2	Name	txtNombre
Textbox3	Name	txtApellido
Textbox4	Name	txtTelefono
Textbox5	Name	txtDireccion
Textbox6	Name	txtBuscar
dataGridView1	Name	dgvContactos
ComboBox1	Name	cboGenero

3. Código del Programa

Funcionalidad 1: Declaración de listas y BindingSource

- ContactManager administra todas las operaciones del sistema.
- BindingSource conecta los datos con el DataGridView.

```
3 referencias
public partial class Form1 : Form
{
    private readonly ContactManager manager = new ContactManager();
    private readonly BindingSource bs = new BindingSource();
}
```

Funcionalidad 2: Clase Contact

La clase representa la estructura principal de cada contacto almacenado en el sistema.

Propiedades del contacto

- ID
- Nombre
- Apellido
- Teléfono
- Dirección
- Género

```
11 referencias
public class Contact
{
    10 referencias
    public int ID { get; set; }
    6 referencias
    public string FirstName { get; set; }
    6 referencias
    public string LastName { get; set; }
    6 referencias
    public string Phone { get; set; }
    6 referencias
    public string Address { get; set; }
    6 referencias
    public string Gender { get; set; }
}
```

Funcionalidad 3: Clase ContactManager

Lista principal de contactos

La lista almacena dinámicamente todos los contactos registrados.

```
2 referencias
private class ContactManager
{
    private readonly List<Contact> contacts = new List<Contact>();
}
```

Funcionalidad 4: Agregar contactos

Método Add

Se valida toda la información antes de almacenar el contacto y evita IDs duplicados. Se garantiza que cada contacto tenga un ID único y datos válidos.

```
1 referencia
public void Add(Contact contact)
{
    Validate(contact);

    if (contacts.Any(c => c.ID == contact.ID))
        throw new InvalidOperationException("Ya existe un contacto con el mismo ID.");

    contacts.Add(contact);
}
```

Funcionalidad 5: Modificar contactos

Método Update

Buscar contacto existente y actualizar información

Permite modificar registros existentes manteniendo el mismo identificador.

```
1 referencia
public void Update(Contact contact)
{
    Validate(contact);

    var existing = contacts.FirstOrDefault(c => c.ID == contact.ID)
        ?? throw new InvalidOperationException("Contacto no encontrado.");

    existing.FirstName = contact.FirstName;
    existing.LastName = contact.LastName;
    existing.Phone = contact.Phone;
    existing.Address = contact.Address;
    existing.Gender = contact.Gender;
}
```

Funcionalidad 6: Eliminar contactos

Método Delete

Elimina completamente un contacto de la lista.

```
1 referencia
public void Delete(int id)
{
    var existing = contacts.FirstOrDefault(c => c.ID == id)
        ?? throw new InvalidOperationException("Contacto no encontrado.");

    contacts.Remove(existing);
}
```

Funcionalidad 7: Búsqueda dinámica

Método Search

Conversión a minúsculas, se usa búsqueda con LINQ*

El sistema permite buscar información parcialmente y en múltiples campos simultáneamente.



```
1 referencia
public IEnumerable<Contact> Search(string term)
{
    if (string.IsNullOrEmpty(term))
        return contacts;

    term = term.ToLower();

    return contacts.Where(c =>
        c.ID.ToString().Contains(term) ||
        (c.FirstName?.ToLower().Contains(term) ?? false) ||
        (c.LastName?.ToLower().Contains(term) ?? false) ||
        (c.Phone?.ToLower().Contains(term) ?? false) ||
        (c.Address?.ToLower().Contains(term) ?? false) ||
        (c.Gender?.ToLower().Contains(term) ?? false)
    );
}
```

Funcionalidad 8: Validación de datos

Método Validate

Verifica si todos los datos están completos. Las validaciones mejoran la integridad y calidad de los datos.

```
2 referencias
private void Validate(Contact contact)
{
    if (contact.ID <= 0)
        throw new ArgumentException("El ID debe ser mayor que cero.");

    if (string.IsNullOrEmpty(contact.FirstName))
        throw new ArgumentException("El nombre es obligatorio.");

    if (string.IsNullOrEmpty(contact.LastName))
        throw new ArgumentException("El apellido es obligatorio.");

    if (string.IsNullOrEmpty(contact.Phone))
        throw new ArgumentException("El teléfono es obligatorio.");

    if (string.IsNullOrEmpty(contact.Address))
        throw new ArgumentException("La dirección es obligatoria.");

    if (string.IsNullOrEmpty(contact.Gender))
        throw new ArgumentException("El género es obligatorio.");
}
```

Funcionalidad 9: Inicialización personalizada

Método InitializeCustom

Se configura completamente la interfaz gráfica del sistema.

Carga de géneros

```
1 referencia
private void InitializeCustom()
{
    cboGenero.Items.AddRange(new[] { "Masculino", "Femenino", "Otro" });
}
```

Configuración del DataGridView

```
dgvContactos.AutoGenerateColumns = true;
dgvContactos.DataSource = bs;
dgvContactos.SelectionChanged += dgvContactos_SelectionChanged;
```

Asignación del BindingSource



```
RefreshGrid();
dgvContactos.Columns["ID"].HeaderText = "ID";
dgvContactos.Columns["FirstName"].HeaderText = "Nombre";
dgvContactos.Columns["LastName"].HeaderText = "Apellido";
dgvContactos.Columns["Phone"].HeaderText = "Teléfono";
dgvContactos.Columns["Address"].HeaderText = "Dirección";
dgvContactos.Columns["Gender"].HeaderText = "Género";
}
```

Funcionalidad 10: Actualización del DataGridView

Método RefreshGrid

Actualiza automáticamente los datos visibles según el filtro de búsqueda.

```
4 referencias
private void RefreshGrid()
{
    bs.DataSource = manager.Search(txtBuscar.Text).ToList();
}
```

Funcionalidad 11: Limpiar campos

Método ClearInputs

Reinicia los controles para facilitar nuevos registros.

```
3 referencias
private void ClearInputs()
{
    txtId.Clear();
    txtNombre.Clear();
    txtApellido.Clear();
    txtTelefono.Clear();
    txtDireccion.Clear();
    cboGenero.SelectedIndex = -1;
}
```

Funcionalidad 12: Construcción de contactos

Método TryBuildContact

Convierte correctamente los datos ingresados en un objeto Contact.

Validación de ID

```
1 referencia
private bool TryBuildContact(out Contact contact)
{
    contact = null;

    if (!int.TryParse(txtId.Text.Trim(), out int id) || id <= 0)
    {
        MessageBox.Show("ID inválido.");
        return false;
    }
}
```

Validación de teléfono

```
if (!long.TryParse(txtTelefono.Text.Trim(), out _))
{
    MessageBox.Show("El teléfono debe ser numérico.");
    return false;
}
```

Creación del objeto



```
contact = new Contact
{
    ID = id,
    FirstName = txtNombre.Text.Trim(),
    LastName = txtApellido.Text.Trim(),
    Phone = txtTelefono.Text.Trim(),
    Address = txtDireccion.Text.Trim(),
    Gender = cboGenero.SelectedItem as string
};

return true;
}
```

Funcionalidad 13: Agregar y modificar desde botones

Se reutiliza lógica mediante programación modular.

Botón Agregar

```
1 referencia
private void btnAgregar_Click(object sender, EventArgs e)
{
    ExecuteAction(manager.Add);
}
```

Formulario para agregar un contacto. Campos de entrada: ID Contacto (321434), Nombre (Fabian), Apellido (Mamani), Teléfono (2343434), Dirección (Aqui), Género (Otro). Botón 'Agregar'.

Botón Modificar

```
1 referencia
private void btnModificar_Click(object sender, EventArgs e)
{
    ExecuteAction(manager.Update);
}
```

Formulario para modificar un contacto. Campos de entrada: ID Contacto (234233), Nombre (Gabriel), Apellido (Azul), Teléfono (4324454554), Dirección (por ahi), Género (Masculino). Tabla de contactos:

ID	Nombre	Apellido	Teléfono	Dirección
234233	Gabriel	Azul	4324454554	por ahi
321434	Fabian	Mamani	2343434	Aqui



ID Contacto: Buscar:

Nombre:

Apellido:

Teléfono:

Dirección:

Género:

ID	Nombre	Apellido	Teléfono
234233	Gabriel	Azul	4324454554
321434	Fabian	Mamani	2343434

ID	Nombre	Apellido	Teléfono	Dirección
234233	Daniela	Mendoza	4324454554	por ahi
321434	Fabian	Mamani	2343434	Aqui

Botón Limpiar

Simplemente limpia los datos que sean puesto en los textbox

```
1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    ClearInputs();
}
```

ID Contacto: Buscar:

Nombre:

Apellido:

Teléfono:

Dirección:

Género:

ID	Nombre	Apellido
234233	Daniela	Mendoza

Formulario de registro de contactos con campos para ID, Nombre, Apellido, Teléfono, Dirección y Género, y un botón para limpiar los datos.

Funcionalidad 14: Ejecución centralizada

Método ExecuteAction

Centraliza procesos comunes como validación, actualización y manejo de errores.



```
2 referencias
private void ExecuteAction(Action<Contact> action)
{
    if (!TryBuildContact(out var contact)) return;

    try
    {
        action(contact);
        RefreshGrid();
        ClearInputs();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Funcionalidad 15: Eliminación desde interfaz

Botón Eliminar

El usuario puede eliminar contactos seleccionados directamente desde la tabla.

```
1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (dgvContactos.SelectedRows.Count == 0)
    {
        MessageBox.Show("Seleccione un contacto.");
        return;
    }

    var row = dgvContactos.SelectedRows[0].DataBoundItem as Contact;

    if (row == null) return;

    try
    {
        manager.Delete(row.ID);
        RefreshGrid();
        ClearInputs();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

The screenshot displays a web application for managing contacts. On the left, there is a form with fields for 'ID' (321434), 'Nombre' (Fabian), 'Apellido' (Mamani), 'Telefono' (2343434), 'Acción' (Aquí), and 'Otro' (dropdown). Below the form are buttons: 'Agregar' (green), 'Modificar' (blue), 'Eliminar' (red), and 'Limpiar' (yellow). On the right, there is a table with columns 'ID', 'Nombre', and 'Apellido'. The table contains two rows: one with ID 234233, Nombre Daniela, and Apellido Mendoza; and another with ID 321434, Nombre Fabian, and Apellido Mamani. The second row is selected. Below the table, there is a search bar and a list of buttons: 'Agregar', 'Modificar', 'Eliminar', and 'Limpiar'.

ID	Nombre	Apellido
234233	Daniela	Mendoza
321434	Fabian	Mamani

Funcionalidad 16: Búsqueda automática

Evento TextChanged



La búsqueda se actualiza automáticamente mientras el usuario escribe.

```
1 referencia
private void txtBuscar_TextChanged(object sender, EventArgs e)
{
    RefreshGrid();
}
```

Funcionalidad 17: Selección de registros

Evento SelectionChanged

Permite cargar automáticamente los datos seleccionados en los controles del formulario.

```
1 referencia
private void dgvContactos_SelectionChanged(object sender, EventArgs e)
{
    if (dgvContactos.SelectedRows.Count == 0) return;

    var row = dgvContactos.SelectedRows[0].DataBoundItem as Contact;

    if (row == null) return;

    txtId.Text = row.ID.ToString();
    txtNombre.Text = row.FirstName;
    txtApellido.Text = row.LastName;
    txtTelefono.Text = row.Phone;
    txtDireccion.Text = row.Address;
    cboGenero.SelectedItem = row.Gender;
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- Sistema que permite gestionar contactos.
- Las operaciones CRUD funcionan con éxito.
- La búsqueda dinámica permite mejorar el uso por el usuario.

¿Que se ha encontrado?

- LINQ facilita el filtrado de la información.
- Las validaciones permiten disminuir errores de ingreso.
- El uso de módulos permite mantener mejor el sistema.
- BindingSource CI facilita la realización de la sincronización entre los datos y la interfaz.

CONCLUSIONES

- El sistema de gestión de contactos se desarrolló totalmente.
- El uso de operaciones CRUD permite realizar administración de registros.
- LINQ permite realizar la búsqueda dinámica.
- Las validaciones permiten una mejora de la seguridad y la integridad de la información.
- El uso de la clase ayuda a la correcta organización del sistema.
- BindingSource y DataGridView permiten realizar una interacción visual con el usuario.
- La reutilización de métodos disminuye la duplicación de código.
- El sistema representa una buena base para aplicaciones de administración más complejas.

RECOMENDACIONES

- Llevar a cabo almacenamiento en base de datos.
- Crear exportación de contactos.
- Incorporar la fotografía de contactos.

WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>



Turrado, J. (2022). Introducción rápida a LINQ con C#: manejar información en memoria nunca fue tan sencillo - campusMVP.es. campusMVP.es. Recuperado de https://www.campusmvp.es/recursos/post/introduccion-rapida-a-linq-con-c-sharp.aspx?srsId=AfmBOogx5_WZg2mtHdV0_YFJiBZsZ5dj_t3Us87MablaeW1k-0upk2xK